Day1_AWK

AWK Pattern Block Questions {

For every question:

Execute the command using employee.txt.

Part A Numerical and String Comparisons

Display the name, department, and salary of employees earning more than 70000.
Display employees whose salary is between 60000 and 80000, including both limits.
Display employees whose employment status is Inactive.
Display employees belonging to the VLSI department.

```
sanjanarao@sanjanarao-VMware-Virtual-Platform:~/Awk$ awk '$4>70000 {print $2, $3, $4}' employee.txt
Alice Security 72000
David VLSI 82000
Frank Security 90000
Henry Networking 76000
John VLSI 78000
Liam Security 85000
sanjanarao@sanjanarao-VMware-Virtual-Platform:~/Awk$ awk '$4 >=60000 && $4<=80000 {print $2}' employee.txt
Alice
Bob
Henry
Irene
John
sanjanarao@sanjanarao-VMware-Virtual-Platform:~/Awk$ awk '$5=="Inactive" {print $2}' employee.txt
Carol
Grace
Kavya
sanjanarao@sanjanarao-VMware-Virtual-Platform:~/Awk$ awk '$3=="VLSI" {print $2}' employee.txt
Bob
David
Grace
John
```

Part B Regular-Expression Patterns

Display all records containing the word Security.
Display employees whose names begin with the letters A, B, or C.
Display employees whose department field does not match Security.

```
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk '$3=="Security"' employee.txt
101 Alice Security 72000 Active
103 Carol Security 48000 Inactive
106 Frank Security 90000 Active
109 Irene Security 68000 Active
112 Liam Security 85000 Active
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk '$2~/^[ABC]/ {print $2}' employee.txt
Alice
Bob
Carol
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk '$3!="Security" {print $2}' employee.txt
Bob
David
Eva
Grace
Henry
John
Kavya
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$
```

Part C Logical Patterns

Display employees who work in the Security department and earn more than 70000.
Display employees who belong to either the VLSI or Networking department.
Display employees who:

Work in the VLSI department
Earn more than 70000
Have Active status

```
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk '$3=="Security" && $4>70000 {print $2}' employee.txt
Alice
Frank
Liam
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk '$3=="VLSI" || $3=="Networking" {print $2}' employee.txt
Bob
David
Eva
Grace
Henry
John
Kavya
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk '$3=="VLSI" && $4>70000 && $5=="Active" {print $0}' employee.txt
104 David VLSI 82000 Active
110 John VLSI 78000 Active
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$
```

Part D Conditional Pattern

Department-dependent salary condition
Apply different salary conditions:
If the employee belongs to Security, select employees earning more than 75000.
Otherwise, select employees earning more than 70000.
Display the employee name, department, and salary.

```
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk '($3=="Security" ? $4>75000: $4>70000) {print $2,$3,$4}' employee.txt
David VLSI 82000
Frank Security 90000
Henry Networking 76000
John VLSI 78000
Liam Security 85000
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$
```

Part E Grouped Patterns

Display active employees who belong to either the Security or VLSI department.
The Active condition must apply to both departments.

Display employees who satisfy one of these conditions:
Work in Security and earn more than 80000
Work in VLSI and earn more than 75000

Display active employees who satisfy one of these conditions:
Work in Security and earn more than 70000
Work in VLSI and earn more than 75000

```
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk '$5=="Active" && ($3=="Security" || $3=="VLSI") {print $0}' employee.txt
101 Alice Security 72000 Active
102 Bob VLSI 65000 Active
104 David VLSI 82000 Active
106 Frank Security 90000 Active
109 Irene Security 68000 Active
110 John VLSI 78000 Active
112 Liam Security 85000 Active
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk '($3=="Security" && $4>80000) || ($3=="VLSI" && $4>75000) {print $0}' employee.txt
104 David VLSI 82000 Active
106 Frank Security 90000 Active
110 John VLSI 78000 Active
112 Liam Security 85000 Active
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk '$5=="Active" && (($3=="Security" && $4>70000)|| ($3=="VLSI" && $4>75000)) {print $0}' employee.txt
101 Alice Security 72000 Active
104 David VLSI 82000 Active
106 Frank Security 90000 Active
110 John VLSI 78000 Active
112 Liam Security 85000 Active
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$
```

Part F Negated Patterns

Display employees whose status is not Active.
Use an exact field comparison because the word Inactive contains the text Active.

Display employees who do not belong to either the Security or VLSI department.

```
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk '$5!="Active" {print $0}' employee.txt
103 Carol Security 48000 Inactive
107 Grace VLSI 45000 Inactive
111 Kavya Networking 42000 Inactive
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk '!($3=="Security" || $3=="VLSI") {print $0}' employee.txt

105 Eva Networking 55000 Active
108 Henry Networking 76000 Active
111 Kavya Networking 42000 Inactive
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$
```

Part G Range Patterns

Display all records beginning with employee ID 103 and ending with employee ID 108. Include both boundary records.

Display all records beginning with David and ending with Irene. Include both boundary records.

Explanation:

- $1 == 103 — starting boundary.
- $1 == 108 — ending boundary.
- The comma , connects the two boundary patterns to create a range.
- {print $0} prints each complete record in the range.

Example syntax:

awk 'start_condition, end_condition {print $0}' employee.txt

```
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk '$1==103,$1==108 {print $0}' employee.txt
103 Carol Security 48000 Inactive
104 David VLSI 82000 Active
105 Eva Networking 55000 Active
106 Frank Security 90000 Active
107 Grace VLSI 45000 Inactive
108 Henry Networking 76000 Active
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk '$2=="David",$2=="Irene" {print $0}' employee.txt

104 David VLSI 82000 Active
105 Eva Networking 55000 Active
106 Frank Security 90000 Active
107 Grace VLSI 45000 Inactive
108 Henry Networking 76000 Active
109 Irene Security 68000 Active
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$
```

Part H Built-In Variables

Display only the first five records. Include the record number in the output.

Display records numbered 4 through 9 using the built-in variable NR.

Display only records containing exactly five fields. Print the record number and the complete record.

Display only odd-numbered records using NR

NR stands for Number of Records. It tells us the current line number being read by AW
For our employee file:

NR	Employee record
1	101 Alice Security 72000 Active
2	102 Bob VLSI 65000 Active
3	103 Carol Security 48000 Inactive
4	104 David VLSI 82000 Active

5 105 Eva Networking 55000 Active

6 106 Frank Security 90000 Active

And it continues until record 12.

```
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk 'NR<=5 {print NR, $0}' employee.txt
1 101 Alice Security 72000 Active
2 102 Bob VLSI 65000 Active
3 103 Carol Security 48000 Inactive
4 104 David VLSI 82000 Active
5 105 Eva Networking 55000 Active
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk 'NR>=4 && NR<=9 {print $0}' employee.txt
104 David VLSI 82000 Active
105 Eva Networking 55000 Active
106 Frank Security 90000 Active
107 Grace VLSI 45000 Inactive
108 Henry Networking 76000 Active
109 Irene Security 68000 Active
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk 'NF == 5 {print NR, $0}' employee.txt
1 101 Alice Security 72000 Active
2 102 Bob VLSI 65000 Active
3 103 Carol Security 48000 Inactive
4 104 David VLSI 82000 Active
5 105 Eva Networking 55000 Active
6 106 Frank Security 90000 Active
7 107 Grace VLSI 45000 Inactive
8 108 Henry Networking 76000 Active
9 109 Irene Security 68000 Active
10 110 John VLSI 78000 Active
11 111 Kavya Networking 42000 Inactive
12 112 Liam Security 85000 Active
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk 'NR % 2 == 1 {print $0}' employee.txt
101 Alice Security 72000 Active
103 Carol Security 48000 Inactive
105 Eva Networking 55000 Active
107 Grace VLSI 45000 Inactive
109 Irene Security 68000 Active
111 Kavya Networking 42000 Inactive
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$
```

Part I Arithmetic Conditions

Assume each employee receives a bonus equal to 10% of the salary. Display employees whose calculated bonus exceeds 7000.

Print:

Employee name

Salary

Calculated bonus

Revised salary

Calculate the revised salary after adding a 12% increment. Display employees whose revised salary exceeds 85000.

Calculate the monthly salary by dividing the annual salary by 12. Display employees whose monthly salary exceeds 6000.

```
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk '$4 * 0.10 > 7000 {print $2, $4, $4 * 0.10}' employee.txt
Alice 72000 7200
David 82000 8200
Frank 90000 9000
Henry 76000 7600
John 78000 7800
Liam 85000 8500
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk '$4 * 1.12 > 85000 {print $2, $4, $4 * 1.12}' employee.txt
David 82000 91840
Frank 90000 100800
Henry 76000 85120
John 78000 87360
Liam 85000 95200
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk '$4 / 12 > 6000 {print $2, $4, $4 / 12}' employee.txt
David 82000 6833.33
Frank 90000 7500
Henry 76000 6333.33
John 78000 6500
Liam 85000 7083.33
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$
CDROM
```

Part J Integrated Challenge

Generate a report containing active employees who satisfy one of these conditions:

Work in Security and earn at least 70000

Work in VLSI and earn at least 75000

Work in Networking and earn at least 70000

The output must contain:

Employee ID

Employee name

Department

Salary

Status

Add a report heading using BEGIN and print the total number of selected employees using END.

Script:

```
BEGIN {
   print "ACTIVE EMPLOYEE REPORT"
   print "ID NAME DEPARTMENT SALARY STATUS"
}
5 == "Active"&&((3 == "Security" && 4 >= 70000)||(3 == "VLSI" && 4 >= 75000)||(3 ==
"Networking" && $4 >= 70000)) {
   print $1, $2, $3, $4, $5
```

```
    count++
}
END {
    print "Total selected employees:", count
}
```

```
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ nano active.awk
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$ awk -f active.awk employee.txt
ACTIVE EMPLOYEE REPORT
ID NAME DEPARTMENT SALARY STATUS
101 Alice Security 72000 Active
104 David VLSI 82000 Active
106 Frank Security 90000 Active
108 Henry Networking 76000 Active
110 John VLSI 78000 Active
112 Liam Security 85000 Active
Total selected employees: 6
ubuntusan@ubuntusan-VMware-Virtual-Platform:~/Desktop/awk$
```